

CONFIDENTIAL

VULNERABILITY ASSESSMENT & PENETRATION TEST

Security Assessment Report

GMR Sunrise — Transformation Maturity Dashboard

Application	GMR Sunrise — Transformation Maturity Dashboard
Version tested	1.0.0
Application type	Frontend-only single-page application (React SPA)
Hosting (as tested)	Azure Static Web Apps (static files + CDN)
Live URL tested	<fill in the deployed Azure URL>
Assessment type	Grey-box — static code & config review, dependency audit, and automated dynamic scan (DAST)
DAST tooling	Snyk API & Web (formerly Probely)
Prepared by	<your name / company>
Prepared for	<customer name>
Assessment dates	11–13 July 2026
Report version	1.0 — <date>

01 Executive summary

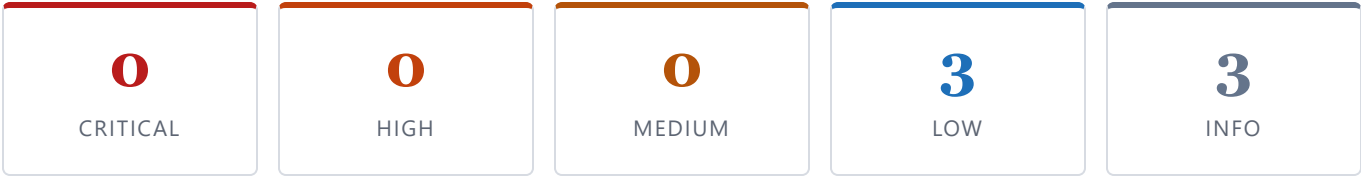
A security assessment was performed on the GMR Sunrise Transformation Maturity Dashboard prior to its deployment into the customer's Microsoft 365 / Azure environment. The assessment combined a **manual review of the complete application source code and configuration**, a **software-composition (dependency) audit**, and an **automated dynamic scan (DAST)** of the deployed application using the commercial platform **Snyk API & Web**.

The application is a **frontend-only single-page application** with **no backend server, no application database, and no server-side code**. All logic executes in the end-user's browser. It is **strictly read-only** — it renders a single Excel workbook from SharePoint and provides **no facility to edit or write data**. It authenticates users against **Microsoft Entra ID** using the **OAuth 2.0 Authorization Code flow with PKCE**, and reads the workbook through the **Microsoft Graph API** using **delegated, read-only** permissions. Because there is no server-side attack surface and no write path, entire vulnerability classes — SQL injection, SSRF, server RCE, insecure direct object references, data tampering — are **not applicable**.

OVERALL
RISK
RATING
LOW

No critical, high, or medium-severity vulnerabilities were identified by source review, dependency audit (0 known vulnerabilities), or the automated dynamic scan (no injection / XSS / sensitive-data / open-redirect findings). Three low-severity and three informational observations were recorded, each a well-understood trade-off with a recommendation.

Findings at a glance



Scope of this rating. This assessment reflects the application as deployed on **Azure Static Web Apps**. If it is later migrated to a different hosting platform, the static/code findings remain valid, but the dynamic checks (§8) should be re-run against the new URL and the platform-portability item **F-06** must be actioned.

02 Scope

In scope

TARGET	DETAIL
Deployed web application	<live URL> (Azure Static Web Apps)
Application source code	Complete React/Vite SPA source (18 files)
Hosting / header configuration	<code>public/staticwebapp.config.json</code>
Authentication & session handling	Microsoft Entra ID + MSAL (PKCE)
Client → Graph data flow	Token handling, scope usage, data rendering
Dependencies (supply chain)	All npm packages shipped in the build

Out of scope

Third-party services operated by Microsoft and Google were **not** tested — Microsoft Entra ID, Microsoft Graph, SharePoint Online, the Azure Static Web Apps platform (except its externally-observable TLS/header configuration), and Google Fonts. The assessment covered **how the application uses** these services, not the services themselves.

03 Methodology

The engagement followed industry-standard methodologies: **OWASP Top 10 (2021)**, the **OWASP Web Security Testing Guide (WSTG)**, and the **OWASP Application Security Verification Standard (ASVS)**. Techniques applied:

TECHNIQUE	DESCRIPTION
SAST	Manual review of all source code and build/hosting configuration.
SCA	Dependency vulnerability audit (<code>npm audit</code>) against the full dependency tree.
DAST	Automated scanning of the deployed app with Snyk API & Web (crawl + active checks, TLS/header inspection).
Manual verification	Authentication flow, client-side storage, clickjacking, header enforcement.

Severity is rated using **CVSS v3.x**: Critical 9.0–10.0, High 7.0–8.9, Medium 4.0–6.9, Low 0.1–3.9, Informational 0.0. Low-severity scores below are taken from the Snyk platform where it reported them.

04 Application architecture

End-user's web browser

React SPA — static files served from Azure Static Web Apps. All logic runs here.

▼ 1. OAuth 2.0 Authorization Code + PKCE ▲ ID + access token

Microsoft Entra ID

OUT OF SCOPE

Sign-in & token issuance (Azure AD).

▼ 2. Bearer access token ▲ Excel workbook (.xlsx)

Microsoft Graph → SharePoint Online

OUT OF SCOPE

Delegated, read-only access to one workbook.

No vendor-controlled server · no database · no third-party analytics · tokens in sessionStorage (cleared on tab close) · read-only.

Security-relevant properties: no vendor server (no server-side surface); all logic client-side (surface = XSS, session/token handling, CSP, dependencies, OAuth config); strictly read-only (no write path, no in-app editing); client/tenant IDs are public identifiers, not secrets — no client secret, password, or API key exists in the application.

05 Findings summary

ID	FINDING	SEVERITY	CVSS	STATUS
F-01	Content-Security-Policy permits <code>'unsafe-inline'</code> for styles	LOW	3.7	Open (accepted)
F-02	Access tokens accessible to same-origin JavaScript (<code>sessionStorage</code>)	LOW	3.1	Open (mitigated)
F-03	Weak TLS cipher suites offered by the hosting platform	LOW	4.2	Open (platform-managed)
F-04	Public configuration values embedded in the JS bundle	INFO	0.0	By design
F-05	Third-party resource load (Google Fonts)	INFO	0.0	Open (optional)
F-06	Security headers/CSP are hosting-platform-specific (Azure SWA)	INFO	0.0	Open (portability)

06 Detailed findings

F-01 CSP permits `'unsafe-inline'` for styles

LOW

STATIC · CONFIRMED BY DAST

CVSS v3.0 3.7 — `AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N`

OWASP A05:2021 – Security Misconfiguration

Component `public/staticwebapp.config.json` → Content-Security-Policy

Source Vendor source review; confirmed by Snyk API & Web DAST (13 Jul 2026)

DESCRIPTION

The CSP is strict for scripts (`script-src 'self'`, `object-src 'none'`, `frame-ancestors 'none'`), but `style-src` includes `'unsafe-inline'`:

```
style-src 'self' 'unsafe-inline' https://fonts.googleapis.com;
```

The Snyk scan independently flagged the same single issue — *"unsafe 'unsafe-inline' in the style-src directive"* — confirming this is the only CSP weakness present.

IMPACT

`'unsafe-inline'` for styles permits injected inline CSS, usable only for limited UI-redress. **Script injection remains fully blocked** (`script-src 'self'`), so JavaScript/XSS execution is not possible via this weakness — practical risk is low.

RECOMMENDATION

`'unsafe-inline'` for styles is **required** by the stack — React inline styles (`style={{...}}`) and Framer Motion animations set inline styles at runtime and cannot use nonces/hashes. Accepted as a documented low residual risk. Removing it would require a significant refactor to eliminate all inline styles.

F-02 Access tokens accessible to same-origin JavaScript

LOW

STATIC

CVSS v3.1 3.1 — AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:N/A:N

OWASP A02:2021 / A07:2021

Component `src/authConfig.js` → `cacheLocation: "sessionStorage"`

DESCRIPTION

MSAL caches tokens in `sessionStorage`, readable by any JavaScript on the app's origin. There is no more secure alternative for a pure SPA without a backend (an HttpOnly cookie would require a server the app deliberately does not have).

IMPACT

A successful XSS attack could read a token and impersonate the user against Graph for the token's lifetime. **Contingent on an XSS vulnerability existing** — none was found (React auto-escaping, no dangerous sinks, and the automated scan reported no XSS).

MITIGATING CONTROLS PRESENT

- `sessionStorage` (not `localStorage`) — cleared when the tab closes (verified in D-7).
- Strict CSP `script-src 'self'`.
- No `eval`, `dangerouslySetInnerHTML`, or `innerHTML` in the source.
- Delegated, read-only Graph scopes — a stolen token cannot write data.

RECOMMENDATION

No change required; this is the recommended pattern for a backend-less SPA. Maintain the strict CSP and safe-rendering practices that keep XSS out.

F-03 Weak TLS cipher suites offered by the hosting platform

LOW

DYNAMIC

CVSS v3.0	4.2 — AV:A/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:N
OWASP	A02:2021 – Cryptographic Failures
Component	Azure Static Web Apps TLS termination (platform-managed)
Source	Snyk API & Web DAST (13 Jul 2026)

DESCRIPTION

The HTTPS endpoint offers some legacy cipher suites that do not provide forward secrecy (TLS_RSA_WITH_AES_128/256_CBC_SHA(256) , TLS_RSA_WITH_AES_128/256_GCM_SHA*) and some CBC-mode ciphers (theoretically exposed to padding-oracle attacks). These are part of the **Azure Static Web Apps TLS configuration, which Microsoft manages** — the application cannot change the cipher list on *.azurestaticapps.net .

IMPACT

Low. Exploitation requires an attacker on the network path (AV:A) able to intercept and downgrade a connection. Modern browsers negotiate the strong ECDHE/GCM and TLS 1.3 suites the same server supports; Qualys SSL Labs rated the endpoint **Grade A** overall (D-2). The weakness applies only to legacy clients that negotiate the older suites.

RECOMMENDATION

Platform-managed — cannot be edited directly on Azure Static Web Apps. To enforce a forward-secrecy-only policy, front the app with **Azure Front Door (Premium)** using a custom TLS security policy (TLS 1.2+, ECDHE/GCM only). Given the overall Grade A rating and low exploitability, may reasonably be **accepted as a low residual risk**.

F-04 Public configuration values embedded in the JS bundle

INFO

STATIC

CVSS 0.0

Component Build output; `.env.example`, `src/authConfig.js`

DESCRIPTION

`VITE_CLIENT_ID` and `VITE_TENANT_ID` are compiled into the JS bundle at build time.

IMPACT

None. These are **public identifiers** present in the client bundle of every SPA. They are not secrets and cannot authenticate without a valid user credential and the PKCE flow. The assessment confirmed no client secret, password, API key, connection string, or token anywhere in the source or bundle.

RECOMMENDATION

No change required. `.env.example` documents this correctly and `.env` is git-ignored.

F-05 Third-party resource load (Google Fonts)

INFO

STATIC / DYNAMIC

CVSS 0.0

Component `index.html` font links; CSP `font-src` / `style-src`

DESCRIPTION

The app loads web fonts from Google (`fonts.googleapis.com`, `fonts.gstatic.com`), both allow-listed in the CSP. End-user browsers therefore make requests to Google for typography.

IMPACT

A minor **privacy** consideration (Google receives request metadata such as IP address), not a security vulnerability. No application or programme data is sent.

RECOMMENDATION

Acceptable for most organisations. If the customer's privacy policy disallows third-party font hosting, self-host the fonts and remove the two Google hosts from the CSP.

F-06 Security headers/CSP are hosting-platform-specific (Azure SWA)

INFO

STATIC · PORTABILITY

CVSS 0.0 — deployment portability

Component `public/staticwebapp.config.json`

DESCRIPTION

The app's entire HTTP-header defence — CSP, HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy, COOP — is delivered through `staticwebapp.config.json`, which is **only honoured by Azure Static Web Apps**. On any other host this file is **silently ignored** and none of these protections apply unless re-implemented using that platform's own mechanism.

IMPACT

No impact while hosted on Azure Static Web Apps (as tested). If migrated elsewhere without re-implementing the headers, the app would lose its CSP, HSTS, clickjacking, and MIME-sniffing protections — a significant regression.

RECOMMENDATION

If hosting is moved, re-create the headers using the target platform's mechanism, then re-verify (§8, D-1). Equivalents:

PLATFORM	HEADER MECHANISM
Azure Static Web Apps (current)	<code>staticwebapp.config.json</code> — in place
Vercel	<code>vercel.json</code> → <code>headers</code>
Netlify	<code>netlify.toml</code> <code>[[headers]]</code> or a <code>_headers</code> file
AWS S3 + CloudFront	CloudFront Response Headers Policy
Azure Blob static website	Azure Front Door / CDN rules (Blob alone cannot set headers)
Cloudflare Pages	<code>_headers</code> file / Transform Rules
GitHub Pages	Not supported — cannot set HTTP headers. Not recommended for this app.
nginx / Apache	<code>add_header</code> / <code>Header set</code> in server config

After any migration, also re-configure the SPA fallback route (deep links → `index.html`) and update the Entra redirect URI to the new domain.

07 Positive security observations

- ✓ **Strictly read-only.** No edit/write capability in the UI and no write path to SharePoint — the dashboard only visualises data.
- ✓ **No dangerous DOM sinks.** No `eval`, `dangerouslySetInnerHTML`, `innerHTML`, or `document.write` anywhere in the source.
- ✓ **Safe rendering.** All external (workbook) data is rendered as auto-escaped text by React.
- ✓ **Zero known dependency vulnerabilities.** `npm audit` — 0 across 138 dependencies.
- ✓ **Clean dynamic scan.** No injection / XSS / open-redirect / sensitive-data findings (Snyk API & Web DAST).
- ✓ **Strict CSP for scripts.** `script-src 'self'`, `object-src 'none'`, `frame-ancestors 'none'`, `base-uri 'self'`.
- ✓ **Full HTTP security-header set — securityheaders.com A+.** HSTS, X-Frame-Options: DENY, X-Content-Type-Options, Referrer-Policy, Permissions-Policy, COOP.
- ✓ **Valid TLS — Qualys SSL Labs Grade A.** TLS 1.3 supported, valid certificate, CAA policy present.
- ✓ **Modern authentication.** OAuth 2.0 Authorization Code + PKCE, no client secret.
- ✓ **Least-privilege data access.** Delegated, read-only Graph scopes: `User.Read`, `Files.Read.All`, `Sites.Read.All`.
- ✓ **Session tokens cleared on sign-out.** Held in `sessionStorage`; verified removed after logout (D-7).
- ✓ **Access control enforced by Entra.** Unassigned users blocked before the app loads (D-6).

08 Dynamic assessment results

Performed against the live URL. D-1/D-2/D-5/D-6/D-7 verified manually and via free scanners; D-3/D-4/D-8/D-9 by the Snyk API & Web automated scan.

#	CHECK	RESULT
D-1	Security-header scan (securityheaders.com)	PASS — A+ , all 6 headers present. 11 Jul 2026.
D-2	TLS/SSL configuration (Qualys SSL Labs)	PASS — Grade A , TLS 1.3, valid cert, CAA present. 11 Jul 2026. (Legacy cipher note → F-03.)
D-3	Crawl + active XSS/injection scan (Snyk)	PASS — no XSS/injection findings. 13 Jul 2026.
D-4	CSP effectiveness / bypass (Snyk)	PASS with note — CSP present; <code>script-src 'self'</code> effective. Only <code>style-src 'unsafe-inline'</code> flagged → F-01. 13 Jul 2026.
D-5	Clickjacking (framing) test	PASS — site refused to load in an iframe; blocked per <code>X-Frame-Options: DENY</code> . 11 Jul 2026.
D-6	Authentication: unassigned account	PASS — unassigned user rejected by Entra (AADSTS50105) before the app loaded. 11 Jul 2026.
D-7	Session: sign-out & token expiry	PASS — after sign-out, returned to sign-in screen; all MSAL tokens cleared from <code>sessionStorage</code> . 11 Jul 2026.
D-8	Sensitive-data-in-response scan (Snyk)	PASS — no information-disclosure findings. 13 Jul 2026.
D-9	OAuth redirect / open-redirect (Snyk)	PASS — no open-redirect findings. 13 Jul 2026.

The Snyk automated scan (13 Jul 2026, unauthenticated, "Normal" profile, "Complete" scope) crawled the application and reported **0 Critical / 0 High / 0 Medium and 2 Low** findings — the CSP item (F-01) and the platform cipher item (F-03), both detailed above. The full Snyk export is attached as **Appendix B**.

09 Conclusion

The GMR Sunrise Transformation Maturity Dashboard presents a **LOW overall security risk** and is considered **suitable for deployment**. No critical, high, or medium-severity vulnerabilities were identified by source review, dependency audit, or the automated dynamic scan. The residual low/informational items are documented, understood trade-offs appropriate to a strictly read-only, backend-less single-page application.

Recommended actions

1. **F-01** (CSP `style-src 'unsafe-inline'`) — accepted low; required by the stack. No action needed unless policy forbids it.
2. **F-03** (platform cipher suites) — Azure-managed; accept, or front with Azure Front Door for a stricter TLS policy if required.
3. Perform the post-deployment CSP verification (confirm the SharePoint download host is allow-listed — see the handover document).
4. **F-05** (Google Fonts) — optional; self-host fonts if policy requires.
5. **F-06** — if hosting ever moves off Azure Static Web Apps, re-implement the security headers and re-run §8 against the new URL.

10 Appendices

Appendix A Tooling — manual source review; `npm audit` (SCA); **Snyk API & Web** (DAST).

Appendix B Snyk API & Web scan report (PDF export attached).

Appendix C Evidence screenshots (D-1 to D-9).

Appendix D References — OWASP Top 10 (2021), OWASP WSTG, OWASP ASVS, CVSS v3.x specification.

This report reflects the security posture of version 1.0.0 at the assessment dates (11–13 July 2026). Any subsequent code, dependency, or configuration change should be re-assessed.